

Design and Implementation of an Air Levitation System using PID Control



Team Members:

Moamen Ahmed	231001011
Marwa Mahgoub	231000083
Adam Hussain	231000724
Saif Abdelmoamen	231001990
Amro Maged	231001357

1. Abstract

This project focuses on the design and construction of a closed-loop control system capable of levitating a ball at a user-defined height within a vertical tube. Using an ultrasonic sensor for feedback and a DC fan for actuation, a Proportional-Integral-Derivative (PID) controller is implemented on an Arduino microcontroller. The project demonstrates the practical application of classical control theory to stabilize an inherently unstable non-linear system.

2. Introduction

Levitation systems are classic benchmarks in control engineering education. They represent a simplified version of complex real-world problems, such as magnetic train levitation, vertical take-off aircraft, and industrial process control.

The objective of this project is to:

1. Construct a physical plant (fan, tube, sensor).
2. Implement a feedback loop using an Arduino.
3. Tune a PID algorithm to minimize error and overshoot.

3. Theoretical Background

3.1 Physical Dynamics

The ball is subject to two primary vertical forces:

1. Gravity (F_g): Acting downwards ($F_g = m \cdot g$).
2. Aerodynamic Drag/Thrust (F_d): Acting upwards, generated by the fan air velocity ($F_d = \frac{1}{2} \rho v^2 C_d A$).

For the ball to hover, F_d must equal F_g . Because the airflow from a fan is turbulent and non-linear, maintaining this balance requires active control.

3.2 The PID Controller

The PID controller calculates an "error" value ($e(t)$) as the difference between a desired setpoint (SP) and a measured process variable (PV). The controller attempts to minimize the error by adjusting the fan speed output ($u(t)$).

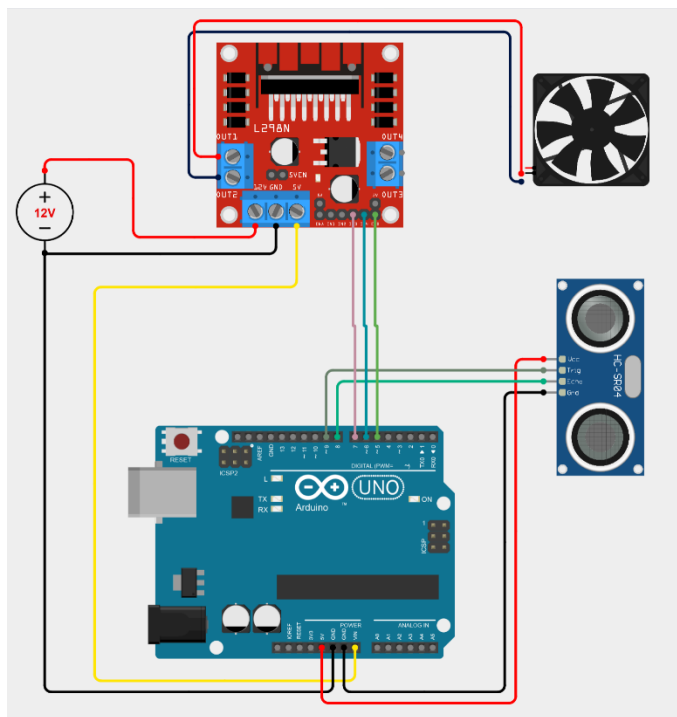
The mathematical equation used is:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

- Proportional (K_p): Provides an immediate output proportional to the current error. Large K_p speeds up response but causes oscillation.
- Integral (K_i): Sums past errors to eliminate steady-state error (offset).
- Derivative (K_d): Predicts future error based on current rate of change, providing a damping effect to reduce overshoot.

4. System Design and Hardware

4.1 Circuit Diagram



4.2 Component Selection

- Microcontroller: Arduino for its ease of PWM generation and sensor interfacing.
- Sensor: HC-SR04 Ultrasonic Module. Chosen for its low cost and 2cm-400cm range. It uses sound waves to measure the distance to the ball from the top of the tube.
- Actuator: 12V DC Fan. High airflow is required to overcome the weight of the ball.
- Driver: L298N H-Bridge Module. Used to amplify the Arduino's 5V logic signal to the 12V required by the fan.

5. Software Implementation

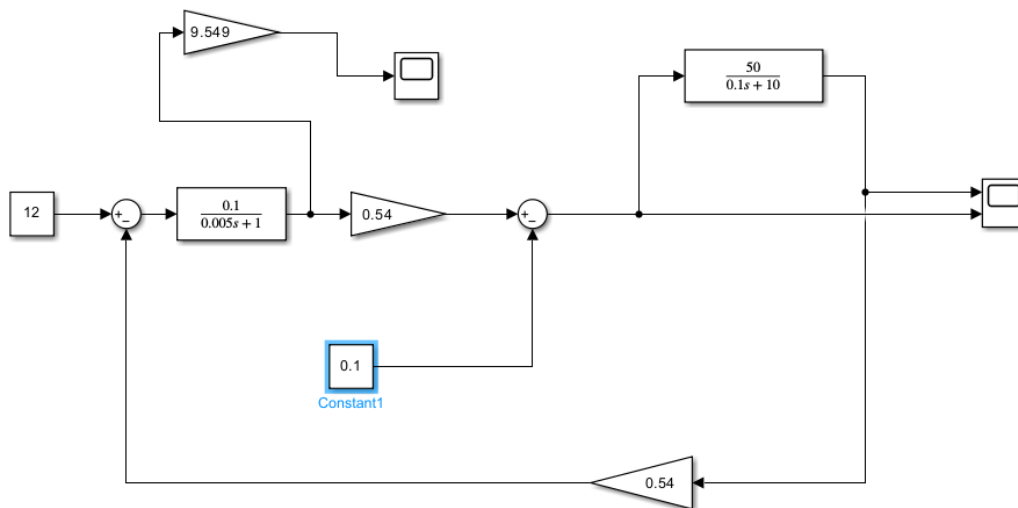
The software logic (Arduino C++) follows these steps:

1. Measure Distance: Trigger the ultrasonic sensor and calculate the distance in cm.
2. Compute Error: Error = Target_Height - Actual_Height.
3. PID Calculation:
 - $P = K_p * \text{Error}$
 - $I = I + (K_i * \text{Error})$ (With integral windup limits)
 - $D = K_d * (\text{Error} - \text{Last_Error})$
 - $\text{PID_Output} = P + I + D$
4. Actuation: Convert the PID output (0-100%) to a PWM signal (0-255) and send it to the fan.

6. Simulation (MATLAB/Simulink)

A simplified Transfer Function model was created to simulate the system response before hardware implementation.

- Plant Model: approximated as a second-order system $G(s) = \frac{K}{s(s+a)}$.
- Results: The simulation showed that a pure Proportional controller results in steady-state error, while adding Integral action eliminates the offset, and Derivative action reduces the settling time.

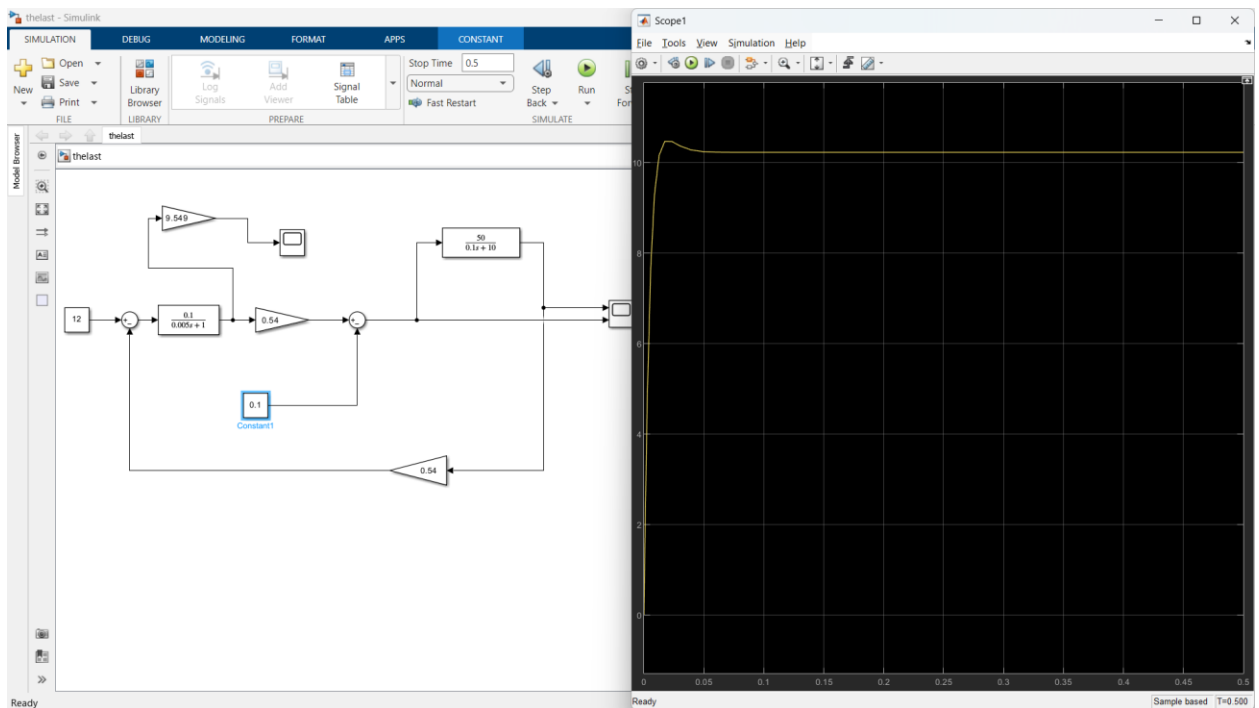
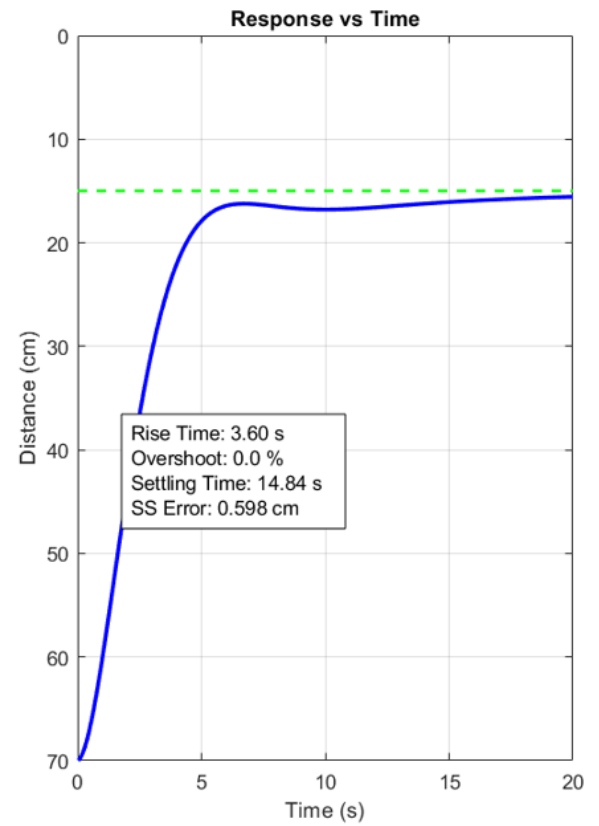
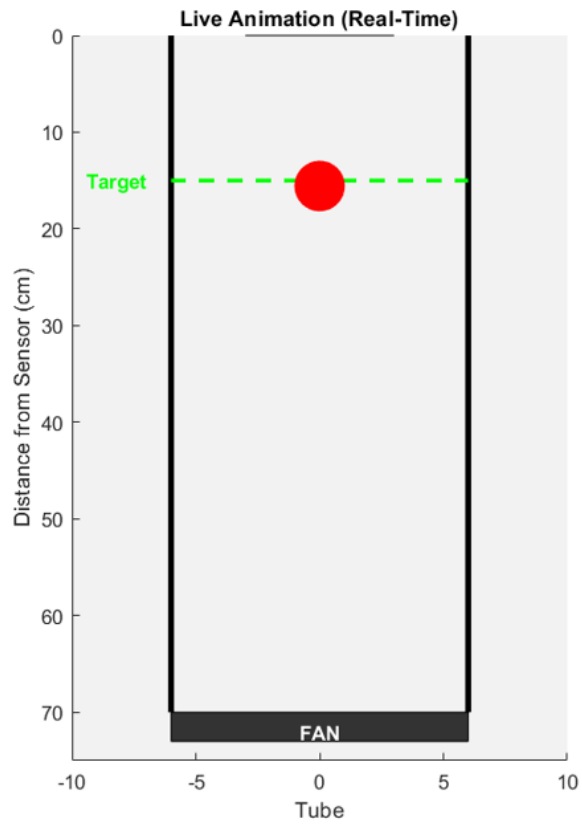


7. Testing and Results

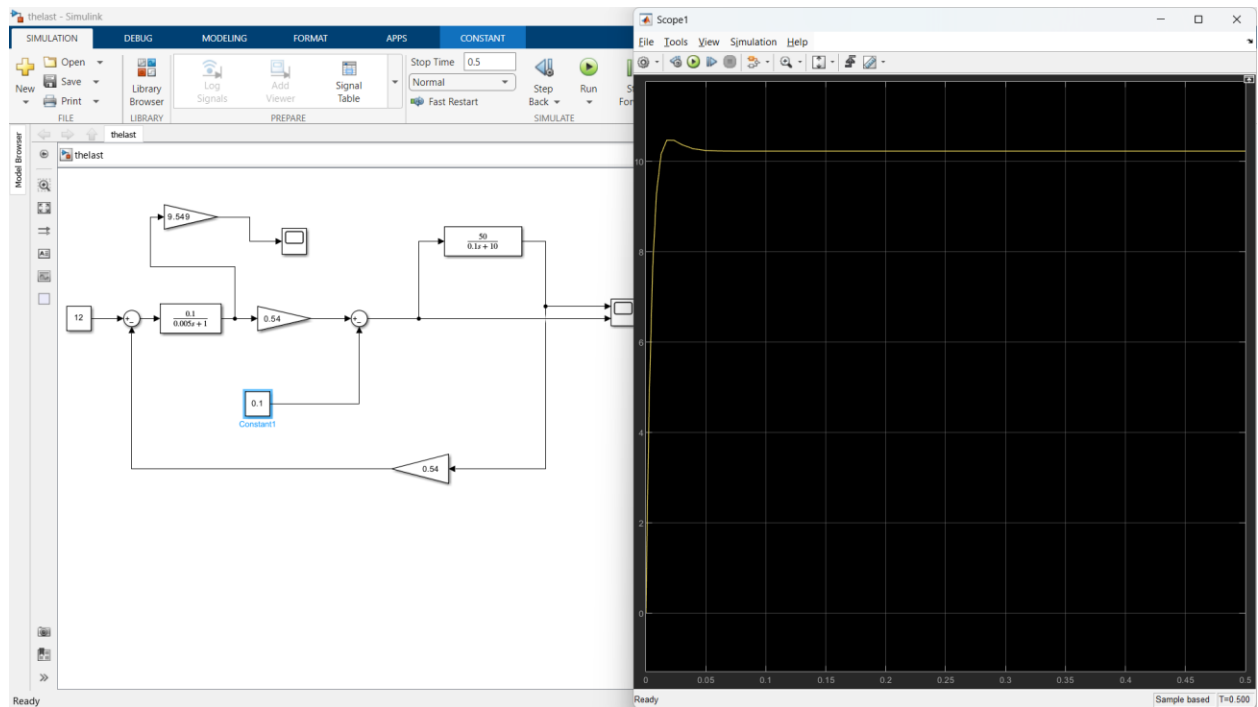
7.1 Tuning Process

The system was tuned experimentally:

1. Kp only: Set to 2.0. The ball oscillated wildly.
2. Adding Kd: Set to 0.5. The oscillations dampened, but the ball hovered below the target.
3. Adding Ki: Set to 0.2. The ball slowly rose to the correct setpoint.



Results of Simulink model of DC motor with Load



Graph of the torque

7.2 Final Performance

The system successfully maintains the ball within $\pm 2\text{cm}$ of the target height. The response time to a step change (changing height from 10cm to 20cm) is approximately 1.5 seconds.

8. Conclusion

The PID Ball Levitation project successfully demonstrates closed-loop control. The primary challenge was the noise from the ultrasonic sensor, which was mitigated using a moving average filter in the software. Future improvements could include using an IR Time-of-Flight sensor for faster feedback loops.

9. Reference

- [mng342](#)